

# **STORE RATING MANAGEMENT SYSTEM**

**Store Rating Management System**

**Submitted By :  
Pratik Khairwar  
PD0523  
PG-DAC**

**Submitted To:  
Hiring team of Roxiler**

**Date:  
6<sup>th</sup> June 2026**

## 2. Abstract / Summary

The Store Rating Management System is a full-stack web application developed using React, Express.js, Node.js, and MySQL. The system allows users to register, log in, rate stores, and manage store-related operations through role-based dashboards. The application provides secure authentication, store management, rating functionality, search and sorting features, and efficient database handling. The project demonstrates frontend-backend integration, REST API development, CRUD operations, JWT authentication, and responsive UI implementation.

## 3. Problem Statement

Many existing systems rely on manual processes for managing stores and customer feedback, which leads to inefficiency, data inconsistency, and poor user experience. There is a need for an automated system that enables users to interact with stores, submit ratings, and manage data securely. This project solves these problems by providing a centralized web-based platform for store management and user rating operations.

## 4. Objectives

- To develop a responsive web application using React.
- To automate store rating and management processes.
- To implement secure user authentication and authorization.
- To provide efficient CRUD operations.
- To create scalable REST APIs using Express.js.
- To maintain organized records using MySQL.
- To implement search, sorting, and filtering functionalities.

## 5. Technologies Used

| Category            | Technology   |
|---------------------|--------------|
| Frontend            | React.js     |
| Backend             | Express.js   |
| Runtime Environment | Node.js      |
| Database            | MySQL        |
| ORM                 | Sequelize    |
| Authentication      | JWT          |
| API Testing         | Postman      |
| Version Control     | Git & GitHub |

## 6. System Architecture / Design

### Frontend Layer

Built using React.js to create interactive and responsive user interfaces.

### Backend Layer

Developed using Express.js to handle API requests, authentication, routing, and business logic.

### Database Layer

Managed using MySQL and Sequelize ORM for storing and managing records.

### Architecture Flow

User → React Frontend → Express.js Backend API → MySQL Database

## 7. Features

- User Registration and Login
- JWT Authentication and Authorization
- Role-Based Access Control
- Store Listing and Management
- Store Rating System
- Admin Dashboard
- Owner Dashboard
- Search and Sorting Functionality
- CRUD Operations
- Error Handling and Validation
- Responsive User Interface

## 8. Project Modules

### 1. Authentication Module

Handles registration, login, JWT authentication, and session management.

### 2. Admin Module

Allows admin users to manage stores, users, and dashboards.

### 3. User Module

Allows users to browse stores and submit ratings.

### 4. Store Owner Module

Allows owners to view ratings and average store ratings.

### 5. API Module

Handles communication between frontend and backend using REST APIs.

## 9. Installation & Setup Guide

### Prerequisites

Install Node.js, MySQL, npm, and Git.

### Clone Repository

```
git clone <repository-link>  
cd project-folder
```

### Install Backend Dependencies

```
npm install
```

### Install Frontend Dependencies

```
cd frontend  
npm install
```

### **Configure Environment Variables**

Create .env file:

DB\_HOST=localhost

DB\_USER=root

DB\_PASSWORD=your\_password

DB\_NAME=store\_rating\_app

JWT\_SECRET=mysecretkey

PORT=5000

### **Create Database**

CREATE DATABASE store\_rating\_app;

### **Start Backend**

npm run dev

### **Start Frontend**

npm run dev

## **10. Folder Structure**

```
project-folder/  
  frontend/  
  src/  
  pages/  
  routes/  
  api/  
  components/  
  backend/  
  controllers/  
  models/  
  routes/  
  middleware/  
  config/  
  README.md  
  package.json  
  server.js
```

## **11. Screenshots / Output**

Include screenshots of:

- Login Page
- Registration Page
- Admin Dashboard
- User Dashboard
- Owner Dashboard
- Store Listing Page
- Rating Submission
- Database Records
- API Response Examples

## **12. Challenges Faced**

- Database connectivity issues
- API integration and routing errors
- Managing JWT authentication
- State management in React
- Handling asynchronous requests
- Role-based route protection
- Sequelize relationship setup

#### **Solutions Implemented**

- Used middleware for authentication and authorization
- Structured backend folders properly
- Improved API validation
- Implemented reusable components
- Optimized database queries

## **13. Future Enhancements**

- Cloud deployment support
- Email notifications
- Advanced analytics dashboard
- Improved UI/UX design
- Mobile application version
- Real-time notifications
- AI-based recommendations

## **14. Conclusion**

The Store Rating Management System successfully demonstrates the implementation of a full-stack web application using React, Express.js, and MySQL. The project automates store management and user rating operations while providing secure authentication, organized database management, and responsive frontend interfaces. This project helped in understanding frontend-backend integration, API development, database management, authentication, and full-stack development practices.

## **15. References**

- React Documentation
- Express.js Documentation
- Node.js Documentation
- MySQL Documentation
- Sequelize Documentation
- Postman Documentation
- JWT Documentation